

PONTIFICIA UNIVERSIDAD JAVERIANA
FACULTAD DE INGENIERÍA



REPORTE DE LABORATORIO .NET
ARQUITECTURA DE SOFTWARE

OCTUBRE 2025

Juan Diego Romero
Kamilt Bejarano Diaz
Pablo Enrique Quintero

Marco Conceptual.....	3
Diseño.....	3
Procedimiento.....	4
Instalación de Visual Studio.....	4
Creación del proyecto en Visual Studio.....	5
Contenedores en Docker.....	6
Migración de la información.....	7
Configuración restante de Visual Studio.....	10
Configuración del explorador de objetos de SQL Server.....	10
Instalación de paquetes NuGet.....	11
Creación de las entidades.....	13
Creación de las interfaces.....	14
Creación de los repositorios.....	15
Creación de los controladores.....	16
Implementación de Swagger.....	18
Ejecución del sistema.....	20
Conclusiones.....	20
Referencias.....	20

Marco Conceptual

El presente taller busca desarrollar una aplicación web basada en el patrón modelo vista controlador (MVC) utilizando el framework .NET y una base de datos de SQL server. Entrando más en detalle en los particulares:

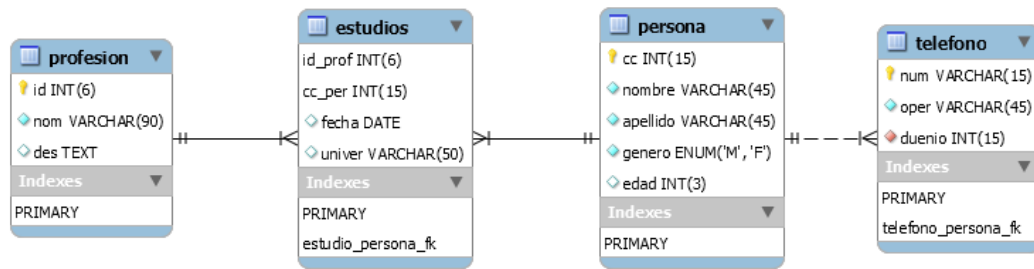
- .NET es un framework de desarrollo multiplataforma de Microsoft que facilita la creación de aplicaciones web modernas, seguras y de alto rendimiento. Incluye un ecosistema integrado para manejar controladores, modelos, vistas y servicios de datos.
- ASP Core MVC implementa el patrón MVC, que divide el sistema en tres capas principales:
 - Modelo: Defina la estructura de datos y las entidades que representan las tablas de la base de datos.
 - Vista: maneja la interfaz de usuario y la presentación de la información.
 - Controlador: actúa como intermediario entre el usuario y el modelo, procesando las solicitudes y devolviendo respuestas o vistas.
- SQL server funciona como el sistema de gestión de bases de datos relacional, encargado de almacenar y administrar la información estructurada.
- Docker: Se utilizó para contener tanto la base de datos como la aplicación, garantizando portabilidad, facilidad de despliegue y consistencia entre entornos de desarrollo y producción.

El propósito de este laboratorio es que el estudiante comprenda el proceso completo de creación de una aplicación web conectada a una base de datos relacional, utilizando herramientas profesionales y un flujo de trabajo similar al profesional mediante el uso de gitHub.

Diseño

El diseño del sistema se basó en una arquitectura por capas siguiendo el patrón MVC. El objetivo fue mantener una estructura organizada, modular y fácil de manejar, donde cada capa cumple una función específica dentro del flujo de la aplicación.

En primer lugar, se diseñó la base de datos en SQL server con cuatro entidades principales siguiendo el modelo dado en la rúbrica del taller:



En la capa de datos, se utilizó Entity framework core, lo que nos permitió generar automáticamente las clases entidad y el contexto de la base de datos.

En la capa de repositorio se implementaron las herramientas para manejar las operaciones de CRUD de cada entidad. Se definieron interfaces con los métodos necesarios y posteriormente se crearon repositorios concretos para su implementación.

En la capa de controladores, se desarrolló un controlador por cada entidad del sistema, persona, teléfono, profesión y estudios. Cada uno se encarga de recibir las solicitudes HTTP, llamar a los métodos del repositorio correspondiente y retornar las respuestas en formato JSON. Esta capa actúa como un puente entre el usuario y la base de datos.

Para la documentación, se integró swagger 3 dentro del proyecto, lo que nos facilita la validación de la API directamente desde el navegador, mostrando de manera visual los controladores, rutas y modelos disponibles.

Finalmente, la aplicación se configuró para ejecutarse dentro de un entorno controlado, utilizando Docker, contenerizado tanto la base de datos como el servicio web. Principalmente, se hizo para asegurar el despliegue consistente en cualquier equipo y para evitar fallas de integración que puede generar la base de datos si se genera de forma local y no desde un contenedor.

Procedimiento

Instalación de Visual Studio

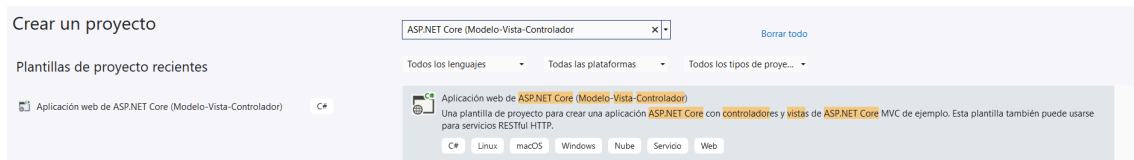
Descargar e instalar Visual Studio Community 2022 desde <https://visualstudio.microsoft.com/es/vs/community/>, junto con los siguientes complementos:

1. Desarrollo ASP.NET y web
2. Almacenamiento y procesamiento de datos

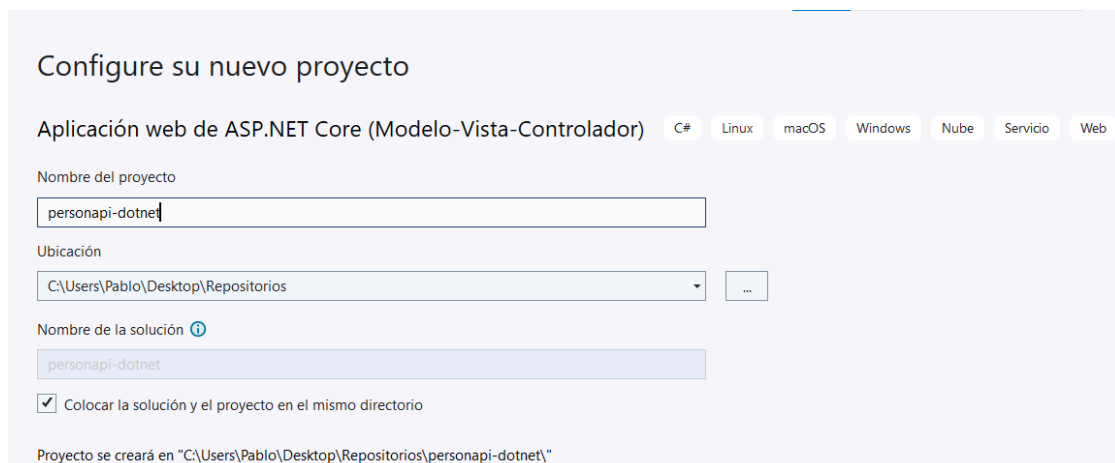
3. Plantillas de proyecto y elementos de .Net Framework
4. Características avanzadas de ASP.NET

Creación del proyecto en Visual Studio

Creamos un nuevo proyecto usando la plantilla *Aplicación web de ASP.NET Core (Modelo-Vista-Controlador)*



Definimos un nombre para nuestro proyecto.



Escogemos nuestra versión de .NET, desactivamos la configuración de HTTPS, y lo creamos.

Información adicional

Aplicación web de ASP.NET Core (Modelo-Vista-Controlador)

C#

Linux

macOS

Windows

Nube

Servicio

Web

Framework

.NET 8.0 (Compatibilidad a largo plazo)

Authentication de campo

Ninguno

☐ Configurar para HTTPS

☐ Habilitar compatibilidad con el contenedor

SO del contenedor

Linux

Tipo de compilación de contenedor

Dockerfile

☐ No usar instrucciones de nivel superior

☐ Inscribirse en la orquestación de .NET Aspire

Versión Aspire

9.5

Contenedores en Docker

Debido al comportamiento de SQL, la base de datos debe de estar Dockerizada, o es imposible de acceder una vez la aplicación se despliegue. Creamos dos contenedores, uno para la base de datos y otro para la página web.

```
1  services:
2    sqlserver:
3      image: mcr.microsoft.com/mssql/server:2022-latest
4      container_name: sqlldb
5      environment:
6        ACCEPT_EULA: "Y"
7        MSSQL_SA_PASSWORD: "DB4dmin!"
8      ports:
9        - "1433:1433"
10     volumes:
11       - sql_data:/var/opt/mssql
12       - ./init-sql:/init-sql
13     restart: always
14
15   personapi-dotnet:
16     build:
17       context: .
18       dockerfile: personapi-dotnet/Dockerfile
19     container_name: webapp
20     ports:
21       - "8080:80"
22     depends_on:
23       - sqlserver
24
25   volumes:
26     sql_data:
```

Al haber configurado Docker en el proyecto, ya tenemos un dockerfile para la página web

```
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 8080

FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
ARG BUILD_CONFIGURATION=Release
WORKDIR /src
COPY ["personapi-dotnet.csproj", "."]
RUN dotnet restore "./personapi-dotnet.csproj"
COPY . .
RUN dotnet build "./personapi-dotnet.csproj" -c $BUILD_CONFIGURATION -o /app/build

FROM build AS publish
ARG BUILD_CONFIGURATION=Release
RUN dotnet publish "./personapi-dotnet.csproj" -c $BUILD_CONFIGURATION -o /app/publish /p:UseAppHost=false

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "personapi-dotnet.dll"]
```

Migración de la información

Dado que la base de datos fue exportada de MySQL, esta debe ser migrada hacia SQL, lo que se puede hacer con **Microsoft SQL Server Migration Assistant (SMAS)**.

Primero se crea una base de datos MySQL temporal para guardar la base de datos.

```
PS C:\Users\Pablo> docker run --name temp-mysql -e MYSQL_ROOT_PASSWORD=TempPass123 -e MYSQL_DATABASE=tempdb -p 3306:3306 -d mysql:8.0
```

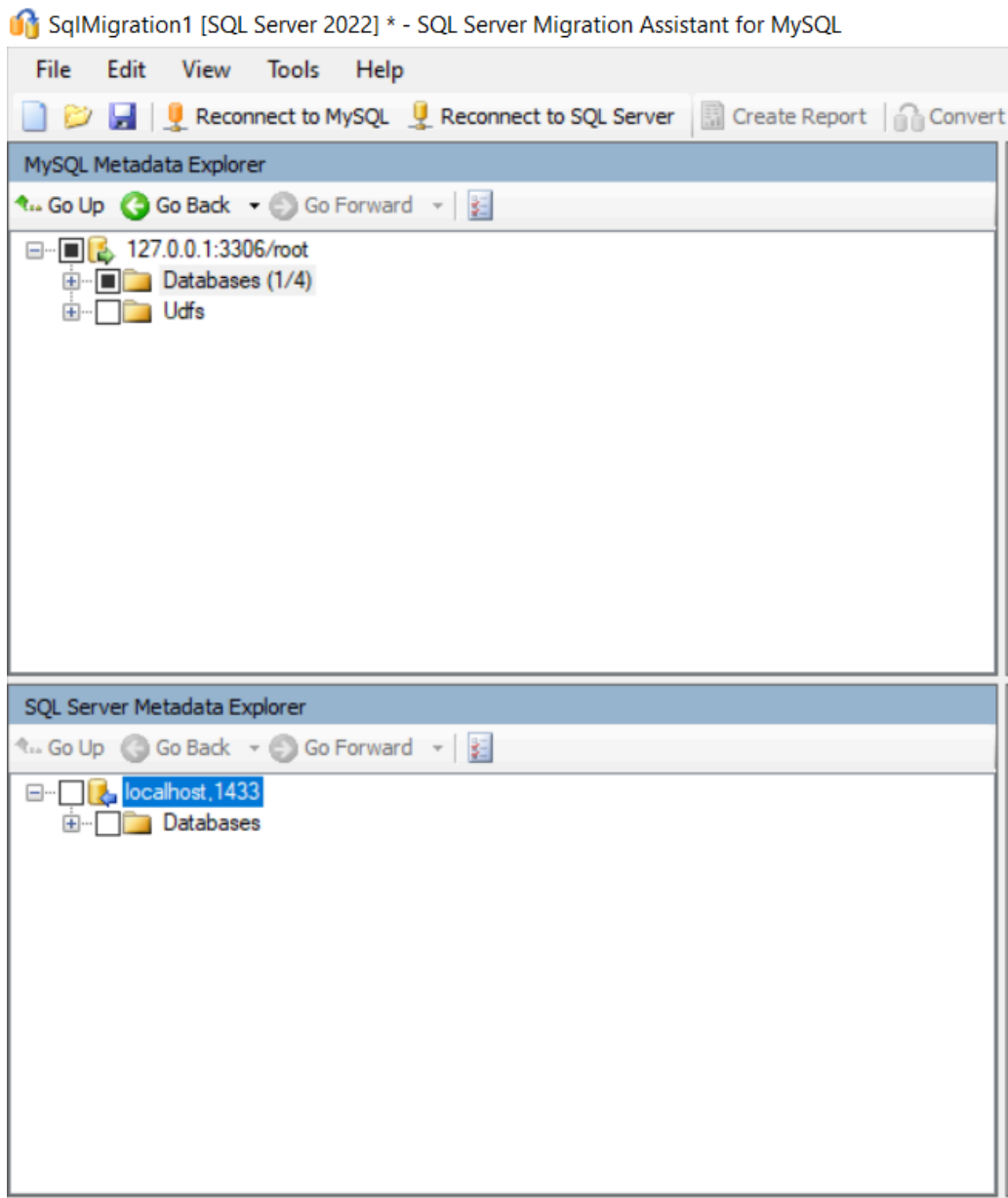
Una vez creada, se carga la base de datos.

```
PS C:\Users\Pablo\Downloads> type .\persona_ddl_mysql.sql | docker exec -i temp-mysql mysql -uroot -pTempPass123 tempdb
```

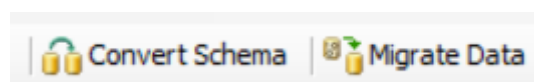
Adicionalmente, con la base de datos SQL en Docker, esta no incluye ninguna base, por lo que debemos crearla primero por medio de un script que ejecutará el archivo .sql:
`sqlcmd -S localhost,1433 -U SA -P DB4dmin! -i ./init-sql/create-db.sql`

```
CREATE DATABASE arq_per_db;
GO
```

Con la base de datos SQL y la Temporal listas en Docker, en SMAS se crea la conexión entre la base de datos temporal y la base de datos SQL.



Utilizamos la opción Convert Schema para migrar el schema de la base de datos MySQL a SQL. Si la base de datos incluye datos, posteriormente utilizamos Migrate Data.



Al hacer click derecho en la base de datos, podemos guardarla como un archivo .sql, el cual agregaremos a la carpeta de inicialización para posteriormente poder cargarla por

medio de otro script una vez se cree el container con la base de datos: `sqlcmd -S localhost,1433 -U SA -P DB4dmin! -i ./init-sql/schema.sql`

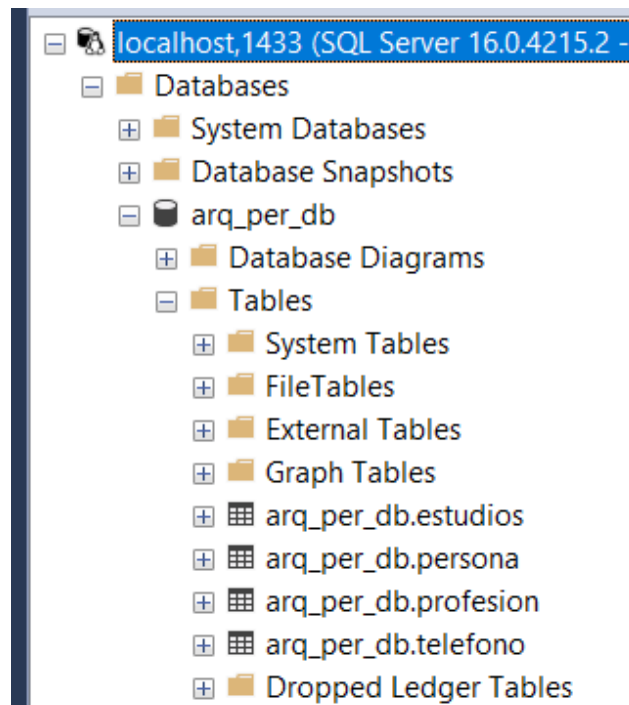
```
USE arq_per_db
GO
IF NOT EXISTS(SELECT * FROM sys.schemas WHERE [name] = N'arq_per_db')
EXEC (N'CREATE SCHEMA arq_per_db')
GO

USE arq_per_db
GO
IF EXISTS (SELECT * FROM sys.objects so JOIN sys.schemas sc ON so.schema_id = sc.schema_id WHERE so.name = N'est
BEGIN

DECLARE @drop_statement nvarchar(500)

DECLARE drop_cursor CURSOR FOR
SELECT 'alter table '+quotename(schema_name(ob.schema_id))+
'.'+quotename(object_name(ob.object_id))+ ' drop constraint ' + quotename(fk.name)
FROM sys.objects ob INNER JOIN sys.foreign_keys fk ON fk.parent_object_id = ob.object_id
WHERE fk.referenced_object_id =
(
SELECT so.object_id
FROM sys.objects so JOIN sys.schemas sc
ON so.schema_id = sc.schema_id
WHERE so.name = N'estudios' AND sc.name = N'arq_per_db' AND type in (N'U')
)
)
```

Una vez terminado el proceso de migración y haber ejecutado el script para las tablas, podemos conectarnos a la base de datos por medio de SQL Server Management para confirmar que la información se encuentra ahí.



Configuración restante de Visual Studio

Configuración del explorador de objetos de SQL Server

Creamos la conexión al Docker con la base de datos.

 Conectar ✕

History

Examinar

 Escriba aquí para filtrar la lista

▶ Local

▶ Red

▶ Azure

Nombre del servidor:

localhost,1433

Autenticación:

Autenticación de SQL Server ▼

Nombre del usuario:

sa

Contraseña:

●●●●●●●●

☐ Recordar contraseña

Nombre de la base de datos:

<predeterminado> ▼

Cifrar:

Mandatory (True) ▼

Confiar en el certificado del servidor:

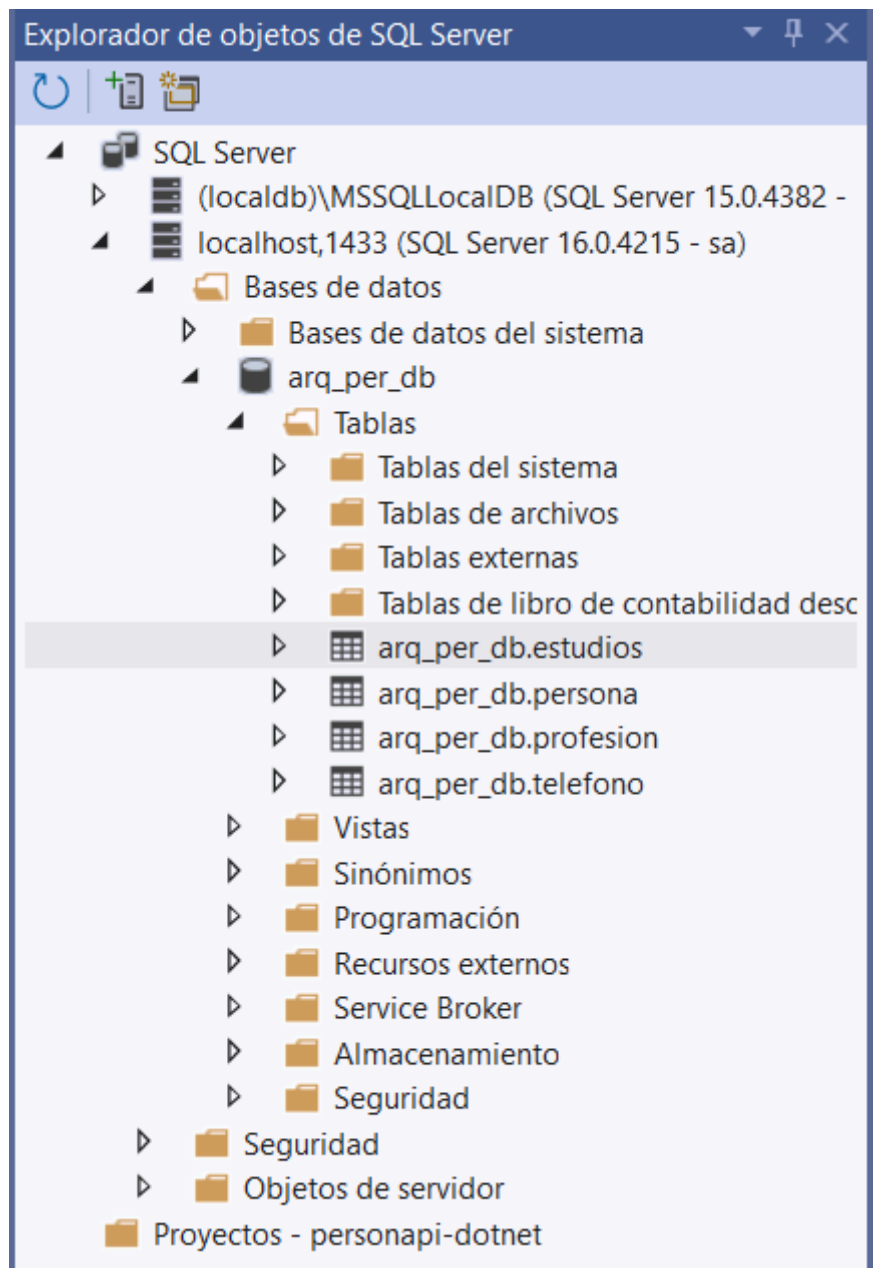
True ▼

[Avanzadas...](#)

Conectar

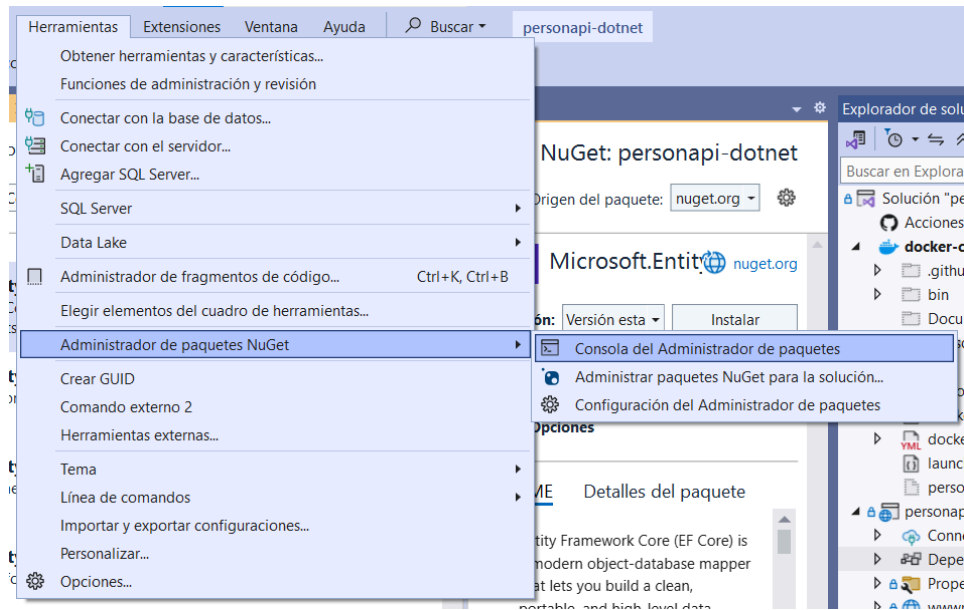
Cancelar

Una vez nos conectemos, podremos ver la base de datos en el explorador.

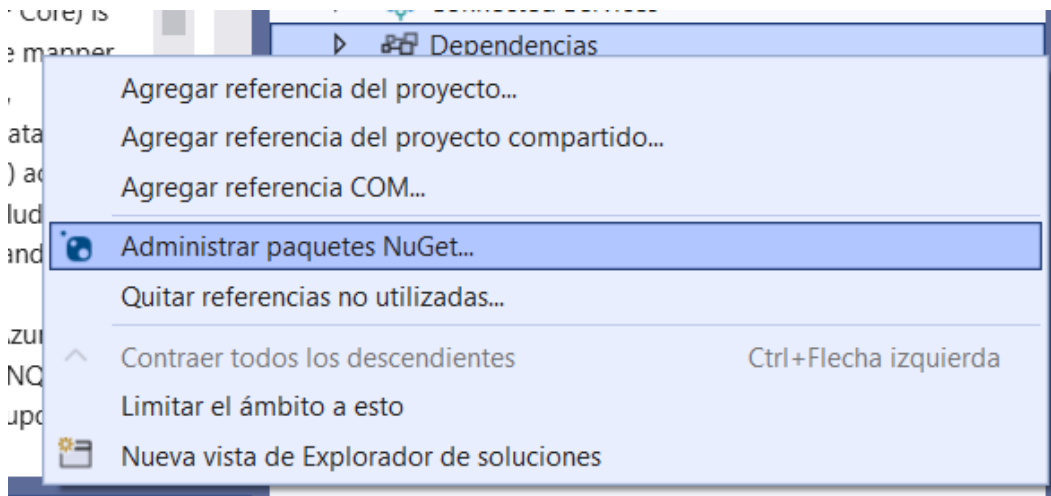


Instalación de paquetes NuGet

Habilitamos la consola de paquetes de NuGet en Herramientas.

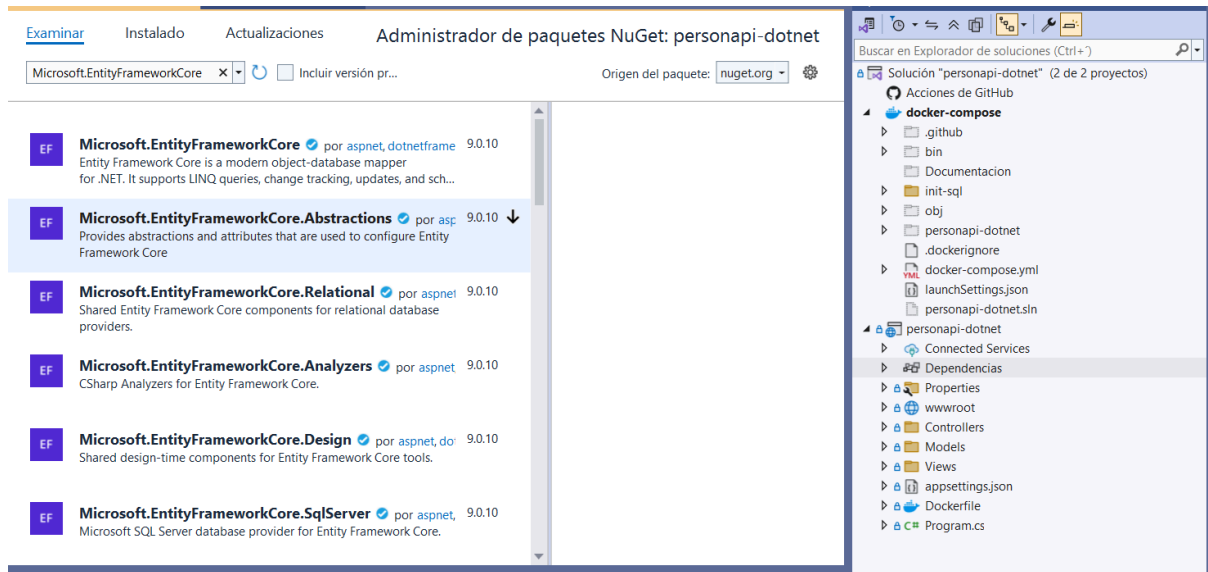


En el explorador de soluciones, hacemos click derecho sobre la carpeta de dependencias y seleccionamos Administrar paquetes NuGet.



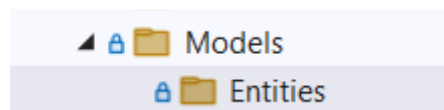
Instalamos los siguientes paquetes:

1. Microsoft.EntityFrameworkCore
2. Microsoft.EntityFrameworkCore.SqlServer
3. Microsoft.EntityFrameworkCore.Tools



Creación de las entidades

Hacer click derecho en la carpeta Models y crear la subcarpeta Entities.



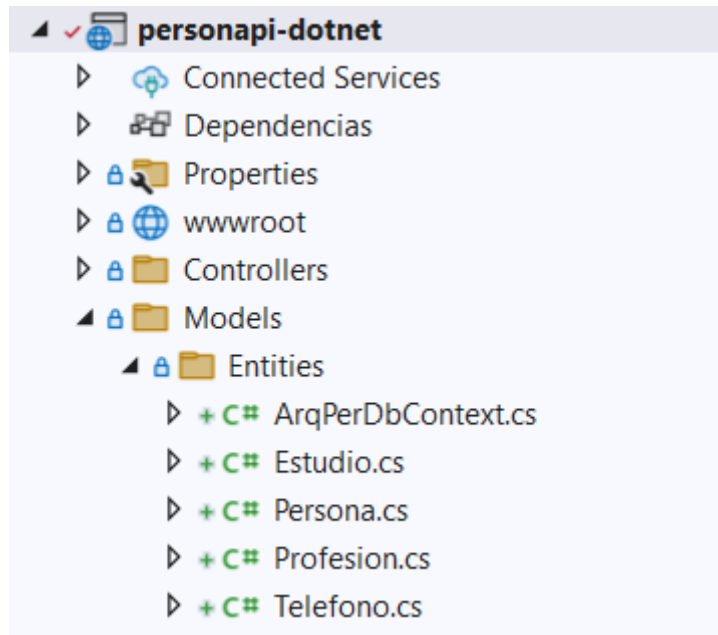
En la consola de administración de paquetes, generar las entidades en base a las tablas de la base de datos con el comando *Scaffold-DbContext*

"Server=localhost,1433;Database=arq_per_db;User

Id=sa;Password=DB4dmin!;TrustServerCertificate=true"

Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models/Entities

```
PM> Scaffold-DbContext "Server=localhost,1433;Database=arq_per_db;User Id=sa;Password=DB4dmin!;TrustServerCertificate=true"
Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models/Entities
Build started...
Build succeeded.
```

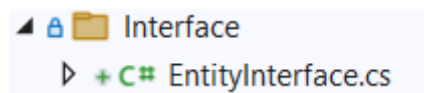


Finalmente, en *appsettings.json* agregamos la cadena *Server=sqlserver,1433;Database=arq_per_db;User Id=sa;Password=DB4dmin!;TrustServerCertificate=true* para realizar la conexión a la base de datos

```
"ConnectionStrings": {  
  "Connection": "Server=sqlserver,1433;Database=arq_per_db;User Id=sa;Password=DB4dmin!;TrustServerCertificate=true;"  
}
```

Creación de las interfaces

Hacemos click derecho dentro de nuestro proyecto para crear una nueva carpeta llamada Entity, sobre esta, volvemos a hacer click derecho para crear nuestra interfaz

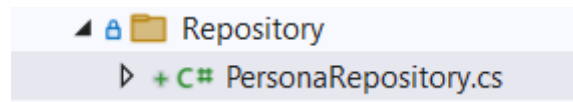


Finalmente, definimos los métodos de nuestra interfaz

```
namespace personapi_dotnet.Interface
{
    1 referencia
    public interface EntityInterface<T>
    {
        2 referencias
        Task<List<T>> findAll();
        1 referencia
        void create(T data);
        1 referencia
        Task<T> findById(int id);
        2 referencias
        void delete(int id);
        1 referencia
        void update(T data);
    }
}
```

Creación de los repositorios

En el root del proyecto hacemos click derecho para crea una nueva carpeta de repositorios, y sobre esta carpeta volvemos a hacer click derecho y creamos el repositorio de cada entidad

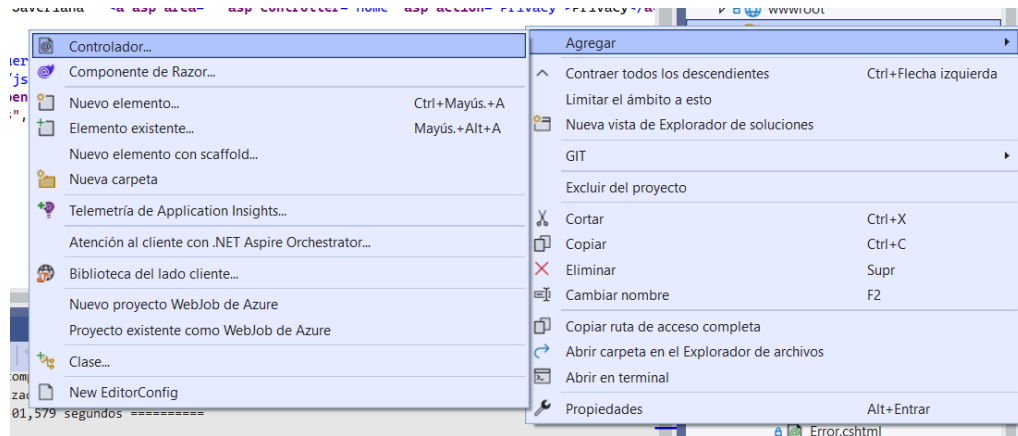


Finalmente, agregamos la implementación de cada método de nuestra interfaz para la entidad

```
public async Task<List<Persona>> findAll()
{
    var personas = await _context.Personas
        .Include(p => p.Telefonos)
        .Include(p => p.Estudios)
        .ToListAsync();
    return personas;
}
```

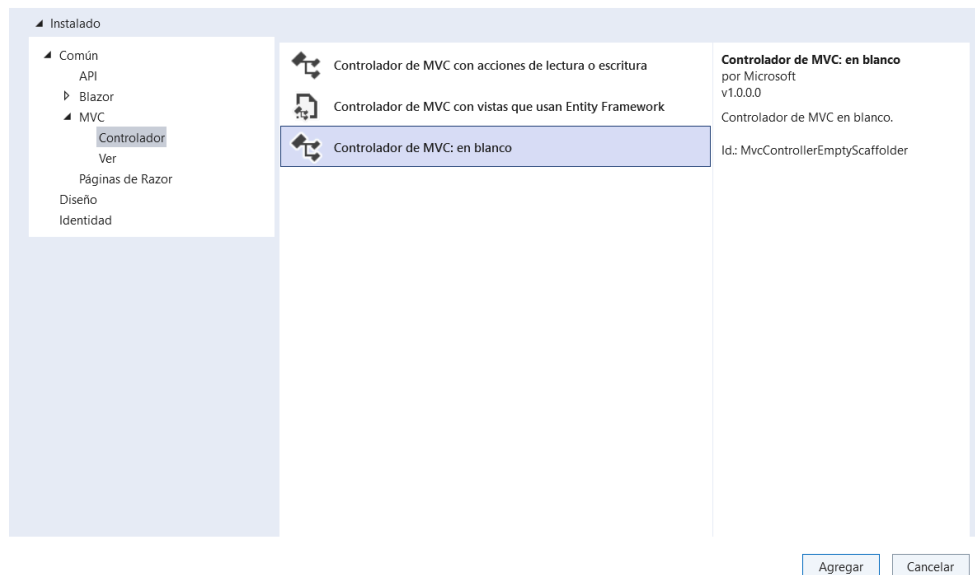
Creación de los controladores

Hacemos click derecho sobre la carpeta Controller, y seleccionamos Agregar Controlador.

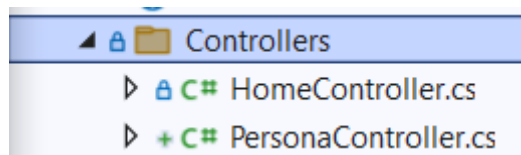
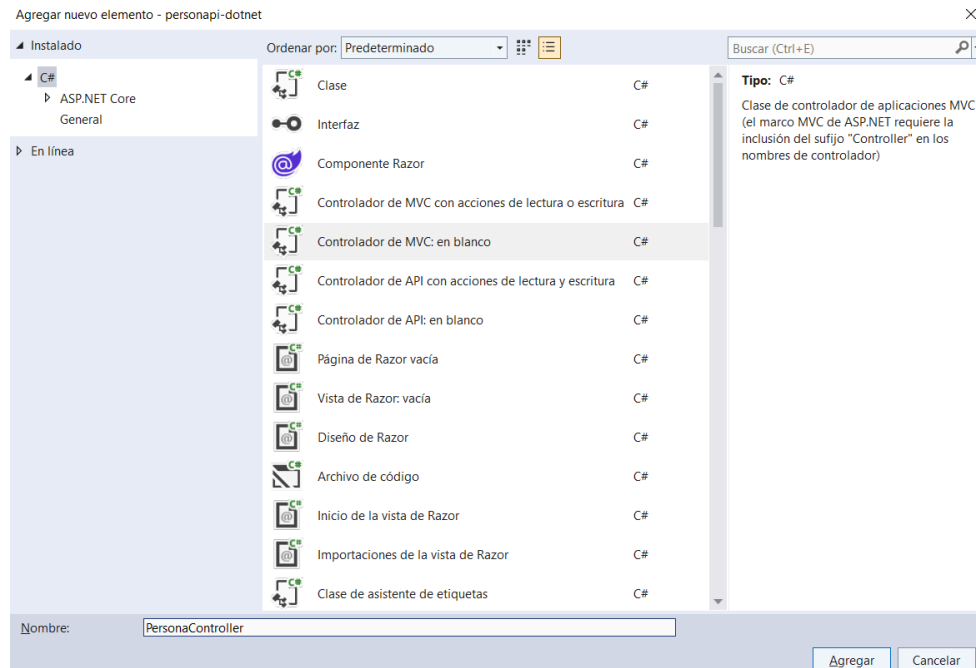


Escogemos la opción de controlador en blanco.

Agregar nuevo elemento con scaffolding



Le damos el nombre correspondiente a la entidad, y lo agregamos.



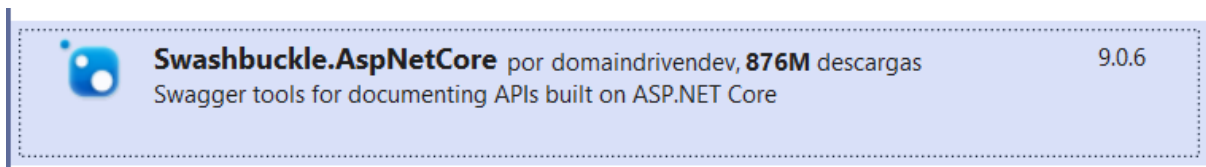
Repetimos este proceso para cada entidad. Desde cada controlador llamamos el repositorio de la entidad

```
[Route("[controller]")]
1 referencia
public class PersonaController : Controller
{
    private readonly IPersonaRepository _repo;
    0 referencias
    public PersonaController(IPersonaRepository repository)
    {
        _repo = repository;
    }

    [HttpGet("Index")]
    3 referencias
    public async Task<IActionResult> Index()
    {
        return View(await _repo.findAll());
    }
}
```

Implementación de Swagger

Instalamos el paquete de NuGet *Swashbuckle.AspNetCore*



Creamos una nueva carpeta llamada Modules/Swagger y creamos el archivo *SwaggerExtensions*



Agregamos el código para la configuración de Swagger

```
using Microsoft.Extensions.DependencyInjection;

public static class SwaggerExtensions
{
    1 referencia
    public static IServiceCollection AddSwagger(this IServiceCollection services)
    {
        services.AddSwaggerGen(c =>
        {
            // Created the Swagger document
            c.SwaggerDoc("v1", new OpenApiInfo
            {
                Version = "Version .Net 9.0.x",
                Title = ".NET PersonB Swagger UI",
                Description = "Arquitectura de Software, Universidad Javeriana 2025 (Juan Diego Romero, Pi
            });

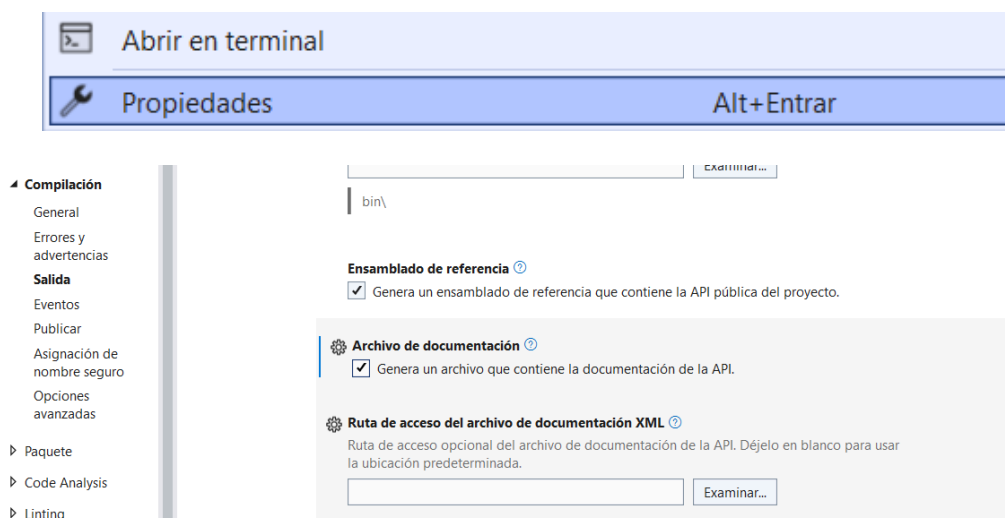
            // form 2 to generate the swagger documentation
            foreach (var name in Directory.GetFiles(AppContext.BaseDirectory, "*.XML", SearchOption.TopDiri
            {
                c.IncludeXmlComments(filePath: name);
            }
        });
    }
}
```

En program.cs, agregamos el servicio de Swagger

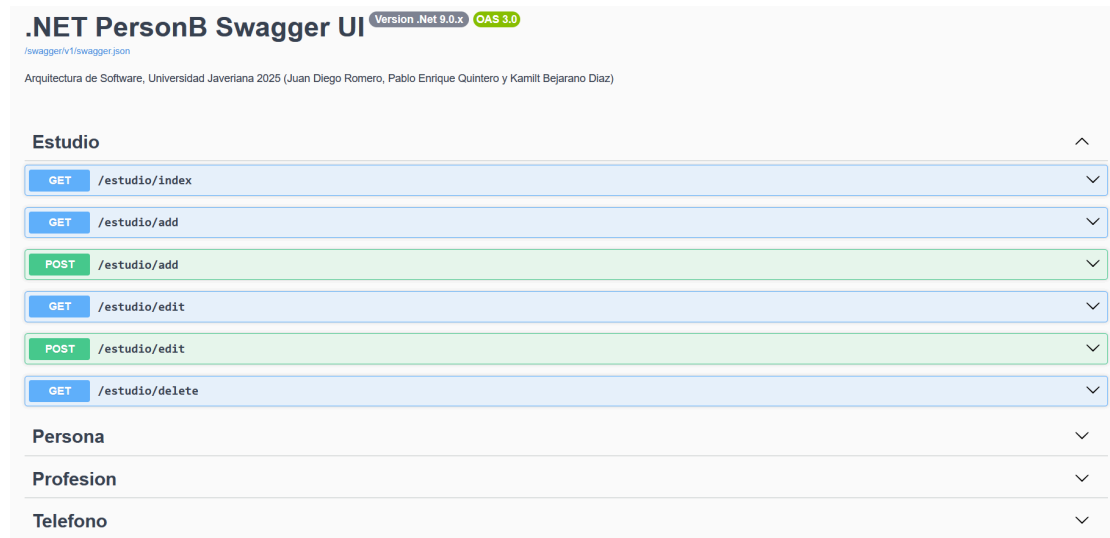
```
builder.Services.AddSwagger();
```

```
app.UseSwagger();
app.UseSwaggerUI(c =>
{
    c.SwaggerEndpoint("/swagger/v1/swagger.json", "Swagger UI Modified V.2");
    c.RoutePrefix = "swagger";
});
```

Abrimos las propiedades del proyecto y en compilación/salida habilitamos el archivo de documentación



La documentación de los endpoints generada por Swagger se puede encontrar en <http://localhost:8080/swagger>



Ejecución del sistema

1. En la terminal, navegar a la carpeta del proyecto con `cd [path]/personapi-dotnet/personapi-dotnet`.
2. Levantar el docker compose con `docker-compose build` y `docker-compose up`.
3. Esperar aproximadamente 1 minuto para que la base de datos SQL se inicialice y acepte conexiones.
4. Ejecutar el siguiente comando para cargar toda la base de datos:
`sqlcmd -S localhost,1433 -U SA -P DB4dmin! -i init.sql`
5. Acceder a la página por `localhost:8080`

Conclusiones

.NET facilita el proceso de crear páginas web gracias a su Wizard, el cual permite crear componentes enteros como controladores y entidades con un solo click, mientras que la integración de SQL Server Manager en su IDE hace que el proceso de trabajar con bases de datos sea más fluido y fácil, eliminando la necesidad de tener el programa ejecutando junto con el editor.

Referencias

https://www.youtube.com/watch?v=6nT-RjMEG0o&ab_channel=hdeleon.net

Arquitectura de Software, Universidad Javeriana

Laboratorio de .NET, Octubre 2025

Pablo Enrique Quintero, Juan Diego Romero, Kamilt Bejarano Diaz

<https://www.youtube.com/watch?v=28LjewDjaz4>

<https://medium.com/@EdsonMZ/c%C3%B3mo-implementar-y-personalizar-swagger-ui-en-una-api-con-net-9-ec965723d852>